

minigrid_rl

March 23, 2026

1 MiniGrid - Navegación con Observación Parcial

Proyecto de Nivel 4 - Reinforcement Learning

Este notebook explora MiniGrid, un entorno donde el agente tiene **observación parcial** - solo ve una porción del mundo. Esto introduce desafíos únicos que no existen en entornos con observación completa.

1.1 Objetivos de Aprendizaje

- Entender la diferencia entre observación completa y parcial
- Analizar la arquitectura CNN personalizada para grids
- Explorar el rol del entropy coefficient en exploración
- Implementar curriculum learning (fácil → difícil)
- Comparar MLP vs CNN para observaciones estructuradas

1.2 1. Setup y Dependencias

```
[1]: # Instalación (descomentar si es necesario)
!pip install minigrid stable-baselines3 gymnasium matplotlib
```

```
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: minigrid in
/home/jeironpro/.local/lib/python3.14/site-packages (3.0.0)
Requirement already satisfied: stable-baselines3 in
/home/jeironpro/.local/lib/python3.14/site-packages (2.7.1)
Requirement already satisfied: gymnasium in
/home/jeironpro/.local/lib/python3.14/site-packages (1.2.2)
Requirement already satisfied: matplotlib in
/home/jeironpro/.local/lib/python3.14/site-packages (3.10.8)
Requirement already satisfied: numpy>=1.18.0 in /usr/lib64/python3.14/site-
packages (from minigrid) (2.3.5)
Requirement already satisfied: pygame>=2.4.0 in
/home/jeironpro/.local/lib/python3.14/site-packages (from minigrid) (2.6.1)
Requirement already satisfied: torch<3.0,>=2.3 in
/home/jeironpro/.local/lib/python3.14/site-packages (from stable-baselines3)
(2.10.0)
Requirement already satisfied: cloudpickle in
/home/jeironpro/.local/lib/python3.14/site-packages (from stable-baselines3)
```

(3.1.2)

Requirement already satisfied: pandas in
/home/jeironpro/.local/lib/python3.14/site-packages (from stable-baselines3)
(3.0.1)

Requirement already satisfied: typing-extensions>=4.3.0 in
/usr/lib/python3.14/site-packages (from gymnasium) (4.15.0)

Requirement already satisfied: farama-notifications>=0.0.1 in
/home/jeironpro/.local/lib/python3.14/site-packages (from gymnasium) (0.0.4)

Requirement already satisfied: filelock in /usr/lib/python3.14/site-packages
(from torch<3.0,>=2.3->stable-baselines3) (3.15.4)

Requirement already satisfied: setuptools in
/home/jeironpro/.local/lib/python3.14/site-packages (from
torch<3.0,>=2.3->stable-baselines3) (80.10.2)

Requirement already satisfied: sympy>=1.13.3 in
/home/jeironpro/.local/lib/python3.14/site-packages (from
torch<3.0,>=2.3->stable-baselines3) (1.14.0)

Requirement already satisfied: networkx>=2.5.1 in
/home/jeironpro/.local/lib/python3.14/site-packages (from
torch<3.0,>=2.3->stable-baselines3) (3.6.1)

Requirement already satisfied: jinja2 in
/home/jeironpro/.local/lib/python3.14/site-packages (from
torch<3.0,>=2.3->stable-baselines3) (3.1.6)

Requirement already satisfied: fsspec>=0.8.5 in
/home/jeironpro/.local/lib/python3.14/site-packages (from
torch<3.0,>=2.3->stable-baselines3) (2026.2.0)

Requirement already satisfied: cuda-bindings==12.9.4 in
/home/jeironpro/.local/lib/python3.14/site-packages (from
torch<3.0,>=2.3->stable-baselines3) (12.9.4)

Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93 in
/home/jeironpro/.local/lib/python3.14/site-packages (from
torch<3.0,>=2.3->stable-baselines3) (12.8.93)

Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90 in
/home/jeironpro/.local/lib/python3.14/site-packages (from
torch<3.0,>=2.3->stable-baselines3) (12.8.90)

Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90 in
/home/jeironpro/.local/lib/python3.14/site-packages (from
torch<3.0,>=2.3->stable-baselines3) (12.8.90)

Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in
/home/jeironpro/.local/lib/python3.14/site-packages (from
torch<3.0,>=2.3->stable-baselines3) (9.10.2.21)

Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1 in
/home/jeironpro/.local/lib/python3.14/site-packages (from
torch<3.0,>=2.3->stable-baselines3) (12.8.4.1)

Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83 in
/home/jeironpro/.local/lib/python3.14/site-packages (from
torch<3.0,>=2.3->stable-baselines3) (11.3.3.83)

Requirement already satisfied: nvidia-curand-cu12==10.3.9.90 in
/home/jeironpro/.local/lib/python3.14/site-packages (from

torch<3.0,>=2.3->stable-baselines3) (10.3.9.90)
 Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90 in
 /home/jeironpro/.local/lib/python3.14/site-packages (from
 torch<3.0,>=2.3->stable-baselines3) (11.7.3.90)
 Requirement already satisfied: nvidia-cusparse-cu12==12.5.8.93 in
 /home/jeironpro/.local/lib/python3.14/site-packages (from
 torch<3.0,>=2.3->stable-baselines3) (12.5.8.93)
 Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in
 /home/jeironpro/.local/lib/python3.14/site-packages (from
 torch<3.0,>=2.3->stable-baselines3) (0.7.1)
 Requirement already satisfied: nvidia-nccl-cu12==2.27.5 in
 /home/jeironpro/.local/lib/python3.14/site-packages (from
 torch<3.0,>=2.3->stable-baselines3) (2.27.5)
 Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in
 /home/jeironpro/.local/lib/python3.14/site-packages (from
 torch<3.0,>=2.3->stable-baselines3) (3.4.5)
 Requirement already satisfied: nvidia-nvtx-cu12==12.8.90 in
 /home/jeironpro/.local/lib/python3.14/site-packages (from
 torch<3.0,>=2.3->stable-baselines3) (12.8.90)
 Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93 in
 /home/jeironpro/.local/lib/python3.14/site-packages (from
 torch<3.0,>=2.3->stable-baselines3) (12.8.93)
 Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3 in
 /home/jeironpro/.local/lib/python3.14/site-packages (from
 torch<3.0,>=2.3->stable-baselines3) (1.13.1.3)
 Requirement already satisfied: triton==3.6.0 in
 /home/jeironpro/.local/lib/python3.14/site-packages (from
 torch<3.0,>=2.3->stable-baselines3) (3.6.0)
 Requirement already satisfied: cuda-pathfinder~=1.1 in
 /home/jeironpro/.local/lib/python3.14/site-packages (from cuda-
 bindings==12.9.4->torch<3.0,>=2.3->stable-baselines3) (1.4.3)
 Requirement already satisfied: contourpy>=1.0.1 in
 /home/jeironpro/.local/lib/python3.14/site-packages (from matplotlib) (1.3.3)
 Requirement already satisfied: cycler>=0.10 in
 /home/jeironpro/.local/lib/python3.14/site-packages (from matplotlib) (0.12.1)
 Requirement already satisfied: fonttools>=4.22.0 in
 /home/jeironpro/.local/lib/python3.14/site-packages (from matplotlib) (4.62.1)
 Requirement already satisfied: kiwisolver>=1.3.1 in
 /home/jeironpro/.local/lib/python3.14/site-packages (from matplotlib) (1.5.0)
 Requirement already satisfied: packaging>=20.0 in
 /home/jeironpro/.local/lib/python3.14/site-packages (from matplotlib) (26.0)
 Requirement already satisfied: pillow>=8 in /usr/lib64/python3.14/site-packages
 (from matplotlib) (11.3.0)
 Requirement already satisfied: pyparsing>=3 in /usr/lib/python3.14/site-packages
 (from matplotlib) (3.1.2)
 Requirement already satisfied: python-dateutil>=2.7 in /usr/lib/python3.14/site-
 packages (from matplotlib) (2.9.0.post0)
 Requirement already satisfied: six>=1.5 in /usr/lib/python3.14/site-packages

```
(from python-dateutil>=2.7->matplotlib) (1.17.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in
/home/jeironpro/.local/lib/python3.14/site-packages (from
sympy>=1.13.3->torch<3.0,>=2.3->stable-baselines3) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in
/home/jeironpro/.local/lib/python3.14/site-packages (from
jinja2->torch<3.0,>=2.3->stable-baselines3) (3.0.3)
```

```
[2]: import numpy as np
import matplotlib.pyplot as plt
from IPython.display import display, HTML
import warnings
warnings.filterwarnings('ignore')

import gymnasium as gym
import minigrid
from minigrid.wrappers import ImgObsWrapper, RGBImgObsWrapper, FullyObsWrapper

import torch
import torch.nn as nn

from stable_baselines3 import PPO, DQN, A2C
from stable_baselines3.common.callbacks import BaseCallback
from stable_baselines3.common.torch_layers import BaseFeaturesExtractor
from stable_baselines3.common.evaluation import evaluate_policy
from stable_baselines3.common.vec_env import DummyVecEnv

print(f"MiniGrid: {minigrid.__version__}")
print(f"PyTorch: {torch.__version__}")
print(f"CUDA disponible: {torch.cuda.is_available()}")
```

```
MiniGrid: 3.0.0
PyTorch: 2.10.0+cu128
CUDA disponible: False
```

1.3 2. Análisis de la Arquitectura

1.3.1 2.1 ¿Qué es la Observación Parcial?

En MiniGrid, el agente **NO ve todo el entorno**. Solo tiene una vista de 7×7 celdas en la dirección que está mirando.

Observación Completa (FullyObs)

```
. . . . .
. . . . .
. . . . .
. . A→. . . .
```

Observación Parcial (Por defecto)

```
?????
?????
. . . .
. . . .
```

```

. . . . . A→. .
. . . . .
. . . . .
. . . . . G
7x7 visible

```

Ve SOLO al frente

Esto crea el problema de **aliasing perceptual**: estados diferentes pueden verse idénticos desde la perspectiva del agente.

```
[3]: # Demostrar la diferencia entre observación parcial y completa
print("="*60)
print("COMPARACIÓN DE OBSERVACIONES")
print("="*60)

# Crear entorno con observación parcial (default)
env_partial = gym.make("MiniGrid-Empty-8x8-v0")
obs_partial, _ = env_partial.reset()

# Crear entorno con observación completa
env_full = gym.make("MiniGrid-Empty-8x8-v0")
env_full = FullyObsWrapper(env_full)
obs_full, _ = env_full.reset()

print(f"\nObservación PARCIAL:")
print(f" Tipo: {type(obs_partial)}")
print(f" Imagen shape: {obs_partial['image'].shape}")
print(f" Descripción: Vista 7x7x3 (solo ve al frente)")

print(f"\nObservación COMPLETA:")
print(f" Tipo: {type(obs_full)}")
print(f" Imagen shape: {obs_full['image'].shape}")
print(f" Descripción: Ve todo el grid")

env_partial.close()
env_full.close()
```

COMPARACIÓN DE OBSERVACIONES

Observación PARCIAL:
 Tipo: <class 'dict'>
 Imagen shape: (7, 7, 3)
 Descripción: Vista 7×7×3 (solo ve al frente)

```
Observación COMPLETA:  
Tipo: <class 'dict'>  
Imagen shape: (8, 8, 3)
```

Descripción: Ve todo el grid

1.3.2 2.2 Estructura de la Observación (7×7×3)

Cada celda tiene 3 canales de información:

Canal	Significado	Valores
0	Tipo de objeto	0=vacío, 1=muro, 2=puerta, 4=llave, 5=pelota, 8=meta
1	Color	0-5 (rojo, verde, azul, púrpura, amarillo, gris)
2	Estado	0=abierto, 1=cerrado, 2=bloqueado (para puertas)

```
[4]: # Visualizar los canales de la observación
env = gym.make("MiniGrid-DoorKey-5x5-v0")
obs, _ = env.reset(seed=42)

fig, axes = plt.subplots(1, 4, figsize=(14, 3))

# Canal 0: Tipo de objeto
im0 = axes[0].imshow(obs['image'][:, :, 0], cmap='viridis')
axes[0].set_title('Canal 0: Tipo de Objeto')
plt.colorbar(im0, ax=axes[0])

# Canal 1: Color
im1 = axes[1].imshow(obs['image'][:, :, 1], cmap='tab10')
axes[1].set_title('Canal 1: Color')
plt.colorbar(im1, ax=axes[1])

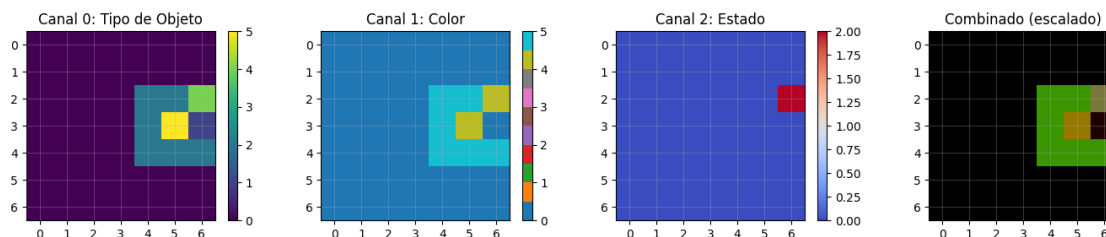
# Canal 2: Estado
im2 = axes[2].imshow(obs['image'][:, :, 2], cmap='coolwarm')
axes[2].set_title('Canal 2: Estado')
plt.colorbar(im2, ax=axes[2])

# Combinado como RGB
axes[3].imshow(obs['image'].astype(np.uint8) * 30) # Escalar para visualizar
axes[3].set_title('Combinado (escalado)')

for ax in axes:
    ax.set_xticks(range(7))
    ax.set_yticks(range(7))
    ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

```
env.close()
```



1.3.3 2.3 Espacio de Acciones

MiniGrid tiene 7 acciones discretas:

Acción	ID	Descripción
Girar izquierda	0	Rotar 90° a la izquierda
Girar derecha	1	Rotar 90° a la derecha
Avanzar	2	Moverse una celda adelante
Recoger	3	Tomar objeto de la celda frontal
Soltar	4	Dejar objeto en celda frontal
Toggle	5	Interactuar (abrir/cerrar puerta)
Done	6	Terminar episodio (raramente usado)

Nota: La mayoría de entornos solo requieren acciones 0-5.

```
[5]: # Demostrar las acciones
from minigrid.core.constants import OBJECT_TO_IDX, COLOR_TO_IDX, STATE_TO_IDX

print("ACCIONES DISPONIBLES")
print("="*40)
acciones = [
    (0, "Girar izquierda", "Rota 90° a la izquierda"),
    (1, "Girar derecha", "Rota 90° a la derecha"),
    (2, "Avanzar", "Mueve una celda adelante"),
    (3, "Recoger", "Toma objeto delante"),
    (4, "Soltar", "Deja objeto delante"),
    (5, "Toggle", "Abre/cierra puerta"),
    (6, "Done", "Termina episodio"),
]
for id, nombre, desc in acciones:
    print(f"  {id}: {nombre:18} - {desc}")

print("\nOBJETOS EN EL GRID")
print("="*40)
for obj, idx in OBJECT_TO_IDX.items():
```

```

    print(f"  {idx}: {obj}")

print("\nCOLORES")
print("="*40)
for color, idx in COLOR_TO_IDX.items():
    print(f"  {idx}: {color}")

```

ACCIONES DISPONIBLES

```

=====
0: Girar izquierda      - Rota 90° a la izquierda
1: Girar derecha        - Rota 90° a la derecha
2: Avanzar              - Mueve una celda adelante
3: Recoger              - Toma objeto delante
4: Soltar               - Deja objeto delante
5: Toggle              - Abre/cierra puerta
6: Done                - Termina episodio

```

OBJETOS EN EL GRID

```

=====
0: unseen
1: empty
2: wall
3: floor
4: door
5: key
6: ball
7: box
8: goal
9: lava
10: agent

```

COLORES

```

=====
0: red
1: green
2: blue
3: purple
4: yellow
5: grey

```

1.3.4 2.4 Función de Recompensa

MiniGrid usa una recompensa **sparse** (escasa):

```

# Recompensa en MiniGrid
if reached_goal:
    reward = 1 - 0.9 * (step_count / max_steps)
else:
    reward = 0

```


- Solo recibe recompensa al llegar a la meta
- Recompensa máxima: ~1.0 (si llega muy rápido)
- Recompensa mínima: ~0.1 (si llega al límite de pasos)
- 0 si no llega

Esto hace el problema **difícil** - el agente debe explorar sin feedback hasta encontrar la meta.

1.3.5 2.5 Arquitectura CNN Personalizada

El código usa una CNN específica para procesar observaciones de MiniGrid.

```
[6]: # Mostrar la arquitectura MinigridCNN del código fuente
class MinigridCNN(BaseFeaturesExtractor):
    """
    CNN para procesar las observaciones de MiniGrid.

    Arquitectura:
    Input: (7, 7, 3) - Vista parcial del agente
    Conv2d(3 → 16, kernel=2) → ReLU → (6, 6, 16)
    Conv2d(16 → 32, kernel=2) → ReLU → (5, 5, 32)
    Conv2d(32 → 64, kernel=2) → ReLU → (4, 4, 64)
    Flatten → 1024
    Linear(1024 → 64) → ReLU
    Output: 64 features
    """
    def __init__(self, observation_space, features_dim=64):
        super().__init__(observation_space, features_dim)

        n_input_channels = observation_space.shape[2] # 3 canales

        self.cnn = nn.Sequential(
            nn.Conv2d(n_input_channels, 16, kernel_size=2, stride=1),
            nn.ReLU(),
            nn.Conv2d(16, 32, kernel_size=2, stride=1),
            nn.ReLU(),
            nn.Conv2d(32, 64, kernel_size=2, stride=1),
            nn.ReLU(),
            nn.Flatten(),
        )

        # Calcular tamaño de salida
        with torch.no_grad():
            sample = torch.zeros(1, n_input_channels, *observation_space.shape[:
↪2])

            n_flatten = self.cnn(sample).shape[1]

        self.linear = nn.Sequential(
            nn.Linear(n_flatten, features_dim),
```

```

        nn.ReLU()
    )

    def forward(self, observations):
        # Cambiar de (B, H, W, C) a (B, C, H, W)
        x = observations.permute(0, 3, 1, 2)
        return self.linear(self.cnn(x))

# Contar parámetros
dummy_space = gym.spaces.Box(low=0, high=1, shape=(7, 7, 3), dtype=np.float32)
cnn = MinigridCNN(dummy_space, features_dim=64)
total_params = sum(p.numel() for p in cnn.parameters())

print("ARQUITECTURA MinigridCNN")
print("="*50)
print(cnn)
print(f"\nTotal parámetros: {total_params:,}")

```

ARQUITECTURA MinigridCNN

=====

```

MinigridCNN(
  (cnn): Sequential(
    (0): Conv2d(3, 16, kernel_size=(2, 2), stride=(1, 1))
    (1): ReLU()
    (2): Conv2d(16, 32, kernel_size=(2, 2), stride=(1, 1))
    (3): ReLU()
    (4): Conv2d(32, 64, kernel_size=(2, 2), stride=(1, 1))
    (5): ReLU()
    (6): Flatten(start_dim=1, end_dim=-1)
  )
  (linear): Sequential(
    (0): Linear(in_features=1024, out_features=64, bias=True)
    (1): ReLU()
  )
)

```

Total parámetros: 76,144

1.3.6 2.6 ¿Por qué CNN y no MLP?

Enfoque	Entrada	Ventajas	Desventajas
MLP	Vector aplanado (147D)	Más rápido, simple	Pierde estructura espacial
CNN	Imagen (7×7×3)	Preserva relaciones espaciales	Más parámetros

La CNN es preferible porque: 1. Detecta patrones locales (muro adelante, puerta a la derecha) 2.

Es invariante a traslaciones parciales 3. Comparte pesos entre posiciones del campo visual

1.4 3. Entornos Disponibles

```
[7]: # Mostrar entornos disponibles organizados por dificultad
ENTORNOS = {
    # Nivel 1: Básicos
    "Empty": ("MiniGrid-Empty-5x5-v0", "Vacío 5×5, ir a meta", " "),
    "Empty8": ("MiniGrid-Empty-8x8-v0", "Vacío 8×8, más exploración", " "),

    # Nivel 2: Con obstáculos
    "FourRooms": ("MiniGrid-FourRooms-v0", "4 habitaciones, pasar entre ellas", " "),
    "SimpleCrossing": ("MiniGrid-SimpleCrossingS9N1-v0", "Cruzar obstáculos", " "),

    # Nivel 3: Con objetos
    "DoorKey": ("MiniGrid-DoorKey-5x5-v0", "Encontrar llave, abrir puerta", " "),
    "DoorKey8": ("MiniGrid-DoorKey-8x8-v0", "DoorKey en grid más grande", " "),
    "Unlock": ("MiniGrid-Unlock-v0", "Solo encontrar y usar llave", " "),

    # Nivel 4: Avanzados
    "LavaCrossing": ("MiniGrid-LavaCrossingS9N1-v0", "Evitar lava mortal", " "),
    "DistShift": ("MiniGrid-DistShift1-v0", "Distribución cambiante", " "),
}

print("ENTORNOS DE MINIGRID")
print("="*70)
print(f"{'Nombre':<15} {'Dificultad':<10} {'Descripción'}")
print("-"*70)
for name, (env_id, desc, diff) in ENTORNOS.items():
    print(f"{'name':<15} {'diff':<10} {'desc'}")
```

ENTORNOS DE MINIGRID

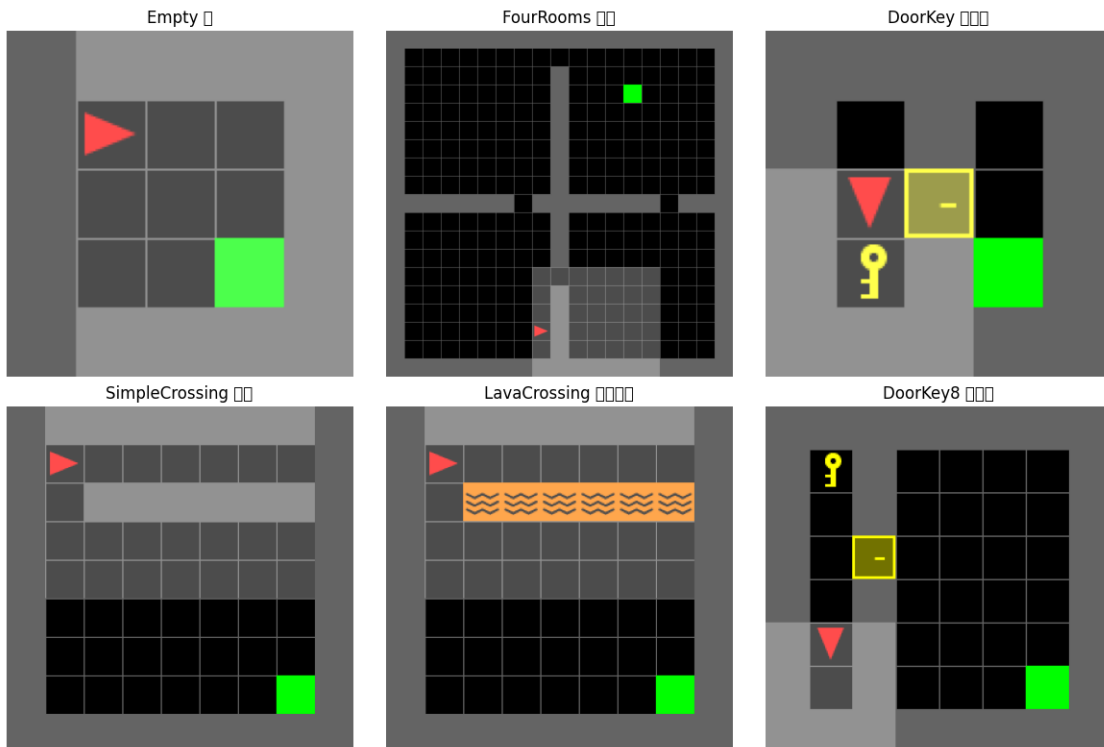
```
=====
Nombre          Dificultad Descripción
-----
Empty           Vacío 5×5, ir a meta
Empty8          Vacío 8×8, más exploración
FourRooms       4 habitaciones, pasar entre ellas
SimpleCrossing  Cruzar obstáculos
DoorKey         Encontrar llave, abrir puerta
DoorKey8        DoorKey en grid más grande
Unlock          Solo encontrar y usar llave
LavaCrossing    Evitar lava mortal
DistShift       Distribución cambiante
```

```
[8]: # Visualizar algunos entornos
fig, axes = plt.subplots(2, 3, figsize=(12, 8))
axes = axes.flatten()

entornos_viz = ["Empty", "FourRooms", "DoorKey", "SimpleCrossing",
↪ "LavaCrossing", "DoorKey8"]

for idx, nombre in enumerate(entornos_viz):
    if nombre in ENTORNOS:
        env_id, desc, diff = ENTORNOS[nombre]
        try:
            env = gym.make(env_id, render_mode="rgb_array")
            env.reset(seed=42)
            img = env.render()
            axes[idx].imshow(img)
            axes[idx].set_title(f"{nombre} {diff}")
            axes[idx].axis('off')
            env.close()
        except Exception as e:
            axes[idx].text(0.5, 0.5, f"Error: {str(e)[:30]}", ha='center',
↪ va='center')
            axes[idx].set_title(nombre)

plt.tight_layout()
plt.show()
```



1.5 4. Código Base para Entrenamiento

```
[9]: # Wrappers del código fuente
class FlatObsWrapper(gym.ObservationWrapper):
    """Aplana la observación para usar con MLP."""
    def __init__(self, env):
        super().__init__(env)
        obs_shape = env.observation_space['image'].shape
        self.observation_space = gym.spaces.Box(
            low=0, high=255,
            shape=(np.prod(obs_shape),),
            dtype=np.float32
        )

    def observation(self, obs):
        return obs['image'].flatten().astype(np.float32) / 255.0

class SimpleObsWrapper(gym.ObservationWrapper):
    """Normaliza la observación de imagen para CNN."""
    def __init__(self, env):
        super().__init__(env)
        self.observation_space = gym.spaces.Box(
            low=0.0, high=1.0,
            shape=env.observation_space['image'].shape,
            dtype=np.float32
        )

    def observation(self, obs):
        return obs['image'].astype(np.float32) / 10.0

class MinigridCallback(BaseCallback):
    """Callback para registrar métricas."""
    def __init__(self, verbose=0):
        super().__init__(verbose)
        self.episode_rewards = []
        self.episode_lengths = []
        self.successes = []

    def _on_step(self) -> bool:
        for info in self.locals.get('infos', []):
            if 'episode' in info:
                self.episode_rewards.append(info['episode']['r'])
```

```

        self.episode_lengths.append(info['episode']['l'])
        self.successes.append(1 if info['episode']['r'] > 0 else 0)
    return True

def crear_entorno(nombre="Empty", wrapper="cnn", render=False):
    """Crea entorno con el wrapper apropiado."""
    env_id = ENTORNOS.get(nombre, (nombre, "", ""))[0] if nombre in ENTORNOS_
    else nombre
    render_mode = "human" if render else None

    env = gym.make(env_id, render_mode=render_mode)

    if wrapper == "flat":
        env = FlatObsWrapper(env)
    else:
        env = SimpleObsWrapper(env)

    return env

def plot_training(callbacks, labels, title="Entrenamiento"):
    """Grafica múltiples entrenamientos."""
    fig, axes = plt.subplots(1, 3, figsize=(15, 4))

    colors = plt.cm.tab10(np.linspace(0, 1, len(callbacks)))

    for callback, label, color in zip(callbacks, labels, colors):
        if not callback.episode_rewards:
            continue

        rewards = callback.episode_rewards
        lengths = callback.episode_lengths
        successes = callback.successes
        window = min(50, len(rewards) // 4) if len(rewards) > 4 else 1

        # Recompensas
        axes[0].plot(rewards, alpha=0.2, color=color)
        if window > 1:
            smoothed = np.convolve(rewards, np.ones(window)/window,
            mode='valid')
            axes[0].plot(range(window-1, len(rewards)), smoothed, color=color,
            label=label, linewidth=2)

        # Longitud
        axes[1].plot(lengths, alpha=0.2, color=color)
        if window > 1:

```

```

        smoothed = np.convolve(lengths, np.ones(window)/window,
↪mode='valid')
        axes[1].plot(range(window-1, len(lengths)), smoothed, color=color,
↪label=label, linewidth=2)

        # Tasa de éxito
        if successes:
            cumsum = np.cumsum(successes)
            rate = cumsum / (np.arange(len(successes)) + 1)
            axes[2].plot(rate, color=color, label=label, linewidth=2)

    axes[0].set_xlabel('Episodio')
    axes[0].set_ylabel('Recompensa')
    axes[0].set_title('Recompensa por Episodio')
    axes[0].legend()
    axes[0].grid(True, alpha=0.3)

    axes[1].set_xlabel('Episodio')
    axes[1].set_ylabel('Pasos')
    axes[1].set_title('Longitud de Episodio')
    axes[1].legend()
    axes[1].grid(True, alpha=0.3)

    axes[2].set_xlabel('Episodio')
    axes[2].set_ylabel('Tasa')
    axes[2].set_title('Tasa de Éxito Acumulada')
    axes[2].set_ylim(0, 1)
    axes[2].legend()
    axes[2].grid(True, alpha=0.3)

    plt.suptitle(title, fontsize=14)
    plt.tight_layout()
    plt.show()

print("Funciones de entrenamiento cargadas.")

```

Funciones de entrenamiento cargadas.

1.6 5. Entrenamiento Demo

```

[10]: # Entrenar agente básico en Empty-5x5
print("="*60)
print("  ENTRENAMIENTO DEMO - Empty 5x5")
print("="*60)

env = crear_entorno("Empty", wrapper="cnn")

```

```

policy_kwargs = {
    "features_extractor_class": MinigridCNN,
    "features_extractor_kwargs": {"features_dim": 64}
}

model = PPO(
    "CnnPolicy",
    env,
    policy_kwargs=policy_kwargs,
    verbose=0,
    learning_rate=0.0003,
    n_steps=128,
    batch_size=64,
    n_epochs=4,
    gamma=0.99,
    ent_coef=0.01,
)

callback = MinigridCallback()

TIMESTEPS = 30000 # Reducido para demo
model.learn(total_timesteps=TIMESTEPS, callback=callback, progress_bar=True)

print(f"\nEpisodios completados: {len(callback.episode_rewards)}")
if callback.episode_rewards:
    print(f"Recompensa promedio (últimos 20): {np.mean(callback.
↪episode_rewards[-20:]):.3f}")
    print(f"Tasa de éxito: {100*sum(callback.successes)/len(callback.successes):
↪.1f}%")

env.close()

```

```

=====
ENTRENAMIENTO DEMO - Empty 5x5
=====

```

```

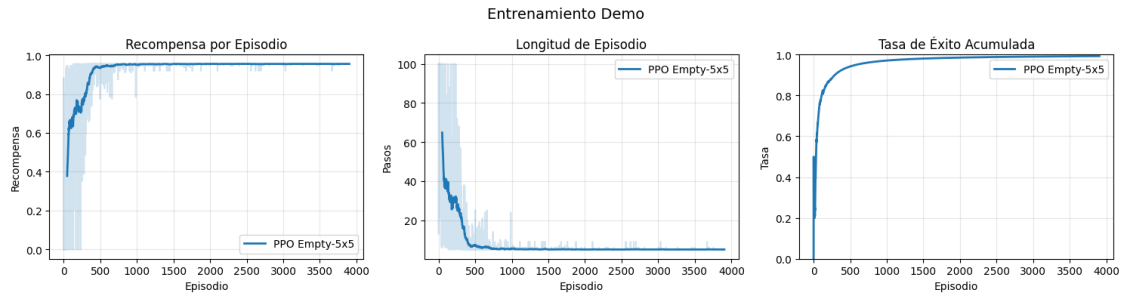
Episodios completados: 3904
Recompensa promedio (últimos 20): 0.955
Tasa de éxito: 99.3%

```

```

[11]: # Visualizar entrenamiento
plot_training([callback], ["PPO Empty-5x5"], "Entrenamiento Demo")

```

1.7 6. Variantes Implementadas

1.7.1 Variante A: MLP vs CNN

```
[12]: def entrenar_mlp_vs_cnn(env_name="Empty8", timesteps=30000):
    """
    Compara MLP (observación aplanada) vs CNN (imagen).

    MLP: Trata la observación como vector de 147 elementos
    CNN: Procesa la imagen 7x7x3 con convoluciones
    """

    print("="*60)
    print(f"  VARIANTE A: MLP vs CNN en {env_name}")
    print("="*60)

    resultados = {}

    # MLP
    print("\n[1/2] Entrenando MLP...")
    env_mlp = crear_entorno(env_name, wrapper="flat")

    model_mlp = PPO(
        "MlpPolicy",
        env_mlp,
        verbose=0,
        learning_rate=0.0003,
        n_steps=128,
        batch_size=64,
        ent_coef=0.01,
    )

    callback_mlp = MinigridCallback()
    model_mlp.learn(total_timesteps=timesteps, callback=callback_mlp,
    ↪ progress_bar=True)
    resultados['MLP'] = callback_mlp
    env_mlp.close()
```

```

# CNN
print("\n[2/2] Entrenando CNN...")
env_cnn = crear_entorno(env_name, wrapper="cnn")

policy_kwargs = {
    "features_extractor_class": MinigridCNN,
    "features_extractor_kwargs": {"features_dim": 64}
}

model_cnn = PPO(
    "CnnPolicy",
    env_cnn,
    policy_kwargs=policy_kwargs,
    verbose=0,
    learning_rate=0.0003,
    n_steps=128,
    batch_size=64,
    ent_coef=0.01,
)

callback_cnn = MinigridCallback()
model_cnn.learn(total_timesteps=timesteps, callback=callback_cnn,
↳progress_bar=True)
resultados['CNN'] = callback_cnn
env_cnn.close()

# Resultados
print("\n" + "="*60)
print("RESULTADOS")
print("="*60)
for nombre, cb in resultados.items():
    if cb.episode_rewards:
        mean_r = np.mean(cb.episode_rewards[-20:])
        success = 100 * sum(cb.successes) / len(cb.successes)
        print(f"{nombre}: Recompensa={mean_r:.3f}, Éxito={success:.1f}%")

return resultados

# Ejecutar comparación
resultados_mlp_cnn = entrenar_mlp_vs_cnn("Empty8", timesteps=30000)
plot_training(
    [resultados_mlp_cnn['MLP'], resultados_mlp_cnn['CNN']],
    ['MLP (147D)', 'CNN (7×7×3)'],
    "Variante A: MLP vs CNN"
)

```

=====

VARIANTE A: MLP vs CNN en Empty8

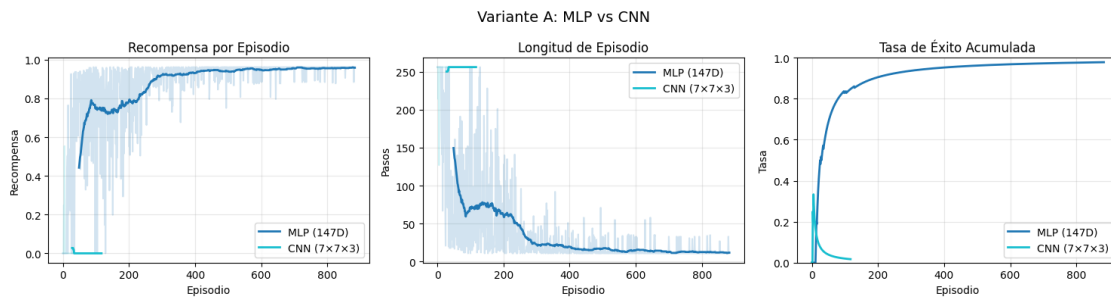
[1/2] Entrenando MLP...

[2/2] Entrenando CNN...

RESULTADOS

MLP: Recompensa=0.957, Éxito=97.9%

CNN: Recompensa=0.000, Éxito=1.7%



1.7.2 Variante B: Curriculum Learning

Entrenar primero en entornos fáciles y transferir a difíciles.

```
[13]: def curriculum_learning(timesteps_per_level=20000):  
    """  
    Curriculum Learning: Entrenar progresivamente en niveles de dificultad.  
  
    Secuencia: Empty5x5 → Empty8x8 → FourRooms → DoorKey5x5  
  
    El modelo mantiene sus pesos entre niveles, permitiendo  
    que el conocimiento de niveles fáciles ayude en los difíciles.  
    """  
    print("="*60)  
    print("  VARIANTE B: CURRICULUM LEARNING")  
    print("="*60)  
  
    curriculum = [  
        ("Empty", "Nivel 1: Empty 5x5"),
```

```

        ("Empty8", "Nivel 2: Empty 8×8"),
        ("FourRooms", "Nivel 3: FourRooms"),
        ("DoorKey", "Nivel 4: DoorKey 5×5"),
    ]

    all_callbacks = []
    model = None

    for idx, (env_name, desc) in enumerate(curriculum):
        print(f"\n[{idx+1}/{len(curriculum)}] {desc}")
        print("-" * 40)

        env = crear_entorno(env_name, wrapper="cnn")

        if model is None:
            # Crear modelo nuevo
            policy_kwargs = {
                "features_extractor_class": MinigridCNN,
                "features_extractor_kwargs": {"features_dim": 64}
            }
            model = PPO(
                "CnnPolicy", env,
                policy_kwargs=policy_kwargs,
                verbose=0,
                learning_rate=0.0003,
                ent_coef=0.01,
            )
        else:
            # Transferir al nuevo entorno
            model.set_env(env)

        callback = MinigridCallback()
        model.learn(total_timesteps=timesteps_per_level, callback=callback,
        ↪progress_bar=True)
        all_callbacks.append((env_name, callback))

        if callback.successes:
            success_rate = 100 * sum(callback.successes) / len(callback.
        ↪successes)
            print(f" → Tasa de éxito: {success_rate:.1f}%")

        env.close()

    return model, all_callbacks

# Comparar: Curriculum vs Entrenar directo en DoorKey
print("\n" + "="*60)

```

```

print("  COMPARACIÓN: CURRICULUM vs DIRECTO")
print("="*60)

# Curriculum
model_curr, callbacks_curr = curriculum_learning(timesteps_per_level=15000)

# Directo (solo DoorKey)
print("\n[DIRECTO] Entrenando solo en DoorKey...")
env_direct = crear_entorno("DoorKey", wrapper="cnn")
policy_kwargs = {
    "features_extractor_class": MinigridCNN,
    "features_extractor_kwargs": {"features_dim": 64}
}
model_direct = PPO("CnnPolicy", env_direct, policy_kwargs=policy_kwargs,
    ↪ verbose=0, ent_coef=0.01)
callback_direct = MinigridCallback()
model_direct.learn(total_timesteps=60000, callback=callback_direct,
    ↪ progress_bar=True) # Mismo total
env_direct.close()

# Comparar en DoorKey
print("\n" + "="*60)
print("RESULTADOS EN DOORKEY")
print("="*60)
curr_doorkey = callbacks_curr[-1][1] # Último nivel del curriculum
print(f"Curriculum: Éxito={100*sum(curr_doorkey.successes)/len(curr_doorkey.
    ↪ successes):.1f}%")
print(f"Directo: Éxito={100*sum(callback_direct.successes)/
    ↪ len(callback_direct.successes):.1f}%")

```

```

=====
COMPARACIÓN: CURRICULUM vs DIRECTO
=====
=====
VARIANTE B: CURRICULUM LEARNING
=====

```

[1/4] Nivel 1: Empty 5×5

→ Tasa de éxito: 92.7%

[2/4] Nivel 2: Empty 8×8

→ Tasa de éxito: 99.9%

[3/4] Nivel 3: FourRooms

→ Tasa de éxito: 19.4%

[4/4] Nivel 4: DoorKey 5×5

→ Tasa de éxito: 10.4%

[DIRECTO] Entrenando solo en DoorKey...

=====

RESULTADOS EN DOORKEY

=====

Curriculum: Éxito=10.4%

Directo: Éxito=96.6%

1.7.3 Variante C: Entropy Coefficient Study

El entropy coefficient controla la exploración del agente.

```
[14]: def entropy_study(env_name="FourRooms", timesteps=30000):  
    """  
    Estudia el efecto del entropy coefficient en la exploración.  
  
    ent_coef alto (0.1): Más exploración, acciones más aleatorias  
    ent_coef bajo (0.0): Menos exploración, acciones más deterministas  
  
    En entornos con recompensa sparse (como MiniGrid),  
    necesitamos suficiente exploración para encontrar la meta.  
    """  
    print("="*60)  
    print(f"  VARIANTE C: ENTROPY COEFFICIENT STUDY")  
    print("="*60)  
  
    ent_coefs = [0.0, 0.01, 0.05, 0.1]  
    resultados = {}  
  
    for ent_coef in ent_coefs:  
        print(f"\nEntrenando con ent_coef={ent_coef}...")
```

```

env = crear_entorno(env_name, wrapper="cnn")

policy_kwargs = {
    "features_extractor_class": MinigridCNN,
    "features_extractor_kwargs": {"features_dim": 64}
}

model = PPO(
    "CnnPolicy", env,
    policy_kwargs=policy_kwargs,
    verbose=0,
    learning_rate=0.0003,
    ent_coef=ent_coef,
)

callback = MinigridCallback()
model.learn(total_timesteps=timesteps, callback=callback,
↳progress_bar=True)

resultados[f"ent={ent_coef}"] = callback
env.close()

return resultados

# Ejecutar estudio
resultados_entropy = entropy_study("FourRooms", timesteps=25000)

# Graficar
callbacks = list(resultados_entropy.values())
labels = list(resultados_entropy.keys())
plot_training(callbacks, labels, "Variante C: Efecto del Entropy Coefficient")

# Tabla de resultados
print("\n" + "="*60)
print("RESUMEN")
print("="*60)
print(f"{'Entropy':<12} {'Éxito':<10} {'Recompensa (últimos 20)'}")
print("-"*50)
for label, cb in resultados_entropy.items():
    if cb.successes:
        success = 100 * sum(cb.successes) / len(cb.successes)
        mean_r = np.mean(cb.episode_rewards[-20:]) if len(cb.episode_rewards)
↳>= 20 else np.mean(cb.episode_rewards)
        print(f"{'label':<12} {'success:>6.1f'}%      {'mean_r:.3f'}")

```

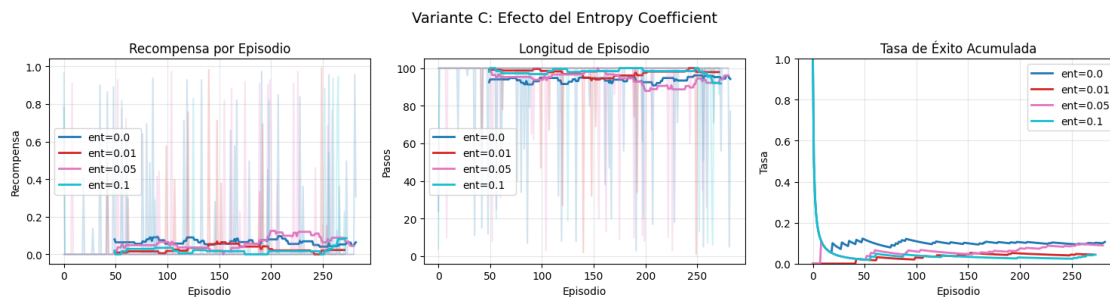
VARIANTE C: ENTROPY COEFFICIENT STUDY

Entrenando con ent_coef=0.0...

Entrenando con ent_coef=0.01...

Entrenando con ent_coef=0.05...

Entrenando con ent_coef=0.1...



RESUMEN

Entropy	Éxito	Recompensa (últimos 20)
---------	-------	-------------------------

ent=0.0	10.6%	0.112
ent=0.01	4.4%	0.008
ent=0.05	8.9%	0.000
ent=0.1	4.4%	0.169

1.7.4 Variante D: Multi-Entorno (Generalización)

```
[15]: def multi_env_training(timesteps=40000):  
    """  
    Entrena en múltiples entornos alternando entre ellos.  
  
    Objetivo: Crear un agente que generalice a nuevos entornos.  
    """
```



```

print("="*60)
print("  VARIANTE D: MULTI-ENTORNO")
print("="*60)

# Entornos de entrenamiento
train_envs = ["Empty", "Empty8", "FourRooms"]
# Entorno de test (no visto durante entrenamiento)
test_env = "SimpleCrossing"

print(f"Entrenamiento en: {train_envs}")
print(f"Evaluación en: {test_env} (no visto)")

# Crear entornos
def make_env(name):
    def _init():
        return crear_entorno(name, wrapper="cnn")
    return _init

# Entrenar rotando entre entornos
all_callbacks = []
model = None

timesteps_per_env = timesteps // len(train_envs)

for env_name in train_envs:
    print(f"\nEntrenando en {env_name}...")
    env = crear_entorno(env_name, wrapper="cnn")

    if model is None:
        policy_kwargs = {
            "features_extractor_class": MinigridCNN,
            "features_extractor_kwargs": {"features_dim": 64}
        }
        model = PPO("CnnPolicy", env, policy_kwargs=policy_kwargs,
↪ verbose=0, ent_coef=0.01)
    else:
        model.set_env(env)

    callback = MinigridCallback()
    model.learn(total_timesteps=timesteps_per_env, callback=callback,
↪ progress_bar=True)
    all_callbacks.append((env_name, callback))
    env.close()

# Evaluar en entorno no visto
print(f"\nEvaluando en {test_env}...")
env_test = crear_entorno(test_env, wrapper="cnn")

```

```

rewards = []
successes = []
for _ in range(50):
    obs, _ = env_test.reset()
    done = False
    total_r = 0
    while not done:
        action, _ = model.predict(obs, deterministic=True)
        obs, r, term, trunc, _ = env_test.step(action)
        total_r += r
        done = term or trunc
    rewards.append(total_r)
    successes.append(1 if total_r > 0 else 0)

env_test.close()

print("\n" + "="*60)
print("RESULTADOS DE GENERALIZACIÓN")
print("="*60)
for name, cb in all_callbacks:
    if cb.successes:
        print(f"{name}: {100*sum(cb.successes)/len(cb.successes):.1f}%\n↳éxito")
    print(f"\n{n{test_env}} (NO VISTO): {100*sum(successes)/len(successes):.1f}%\n↳éxito")
    print(f"Recompensa promedio en test: {np.mean(rewards):.3f}")

return model, all_callbacks, (rewards, successes)

# Ejecutar
model_multi, callbacks_multi, test_results = multi_env_training(timesteps=30000)

```

```

=====
VARIANTE D: MULTI-ENTORNO
=====
Entrenamiento en: ['Empty', 'Empty8', 'FourRooms']
Evaluación en: SimpleCrossing (no visto)

Entrenando en Empty...

```

```

Entrenando en Empty8...

```

Entrenando en FourRooms...

Evaluando en SimpleCrossing...

=====

RESULTADOS DE GENERALIZACIÓN

=====

Empty: 79.2% éxito
Empty8: 99.1% éxito
FourRooms: 19.1% éxito

SimpleCrossing (NO VISTO): 0.0% éxito
Recompensa promedio en test: 0.000

1.7.5 Variante E: Memoria (LSTM) - Conceptual

Para tareas que requieren memoria (recordar dónde está la llave), se necesita arquitectura recurrente.

```
[16]: # Nota: RecurrentPPO requiere sb3-contrib
      # !pip install sb3-contrib

      try:
          from sb3_contrib import RecurrentPPO
          SB3_CONTRIB_AVAILABLE = True
      except ImportError:
          SB3_CONTRIB_AVAILABLE = False
          print("sb3-contrib no instalado. Instalar con: pip install sb3-contrib")

      def entrenar_con_memoria(env_name="DoorKey", timesteps=50000):
          """
          Entrena con LSTM para tareas que requieren memoria.

          En DoorKey, el agente debe:
          1. Encontrar la llave
          2. Recordar dónde está la puerta
          3. Volver y abrirla

          Sin memoria, el agente "olvida" dónde vio la puerta.
          """
          if not SB3_CONTRIB_AVAILABLE:
              print("Requiere sb3-contrib. Ejemplo conceptual:")
              print("""
              from sb3_contrib import RecurrentPPO

              model = RecurrentPPO(
```

```

        "MlpLstmPolicy", # Política con LSTM
        env,
        verbose=1,
        n_steps=128,
        batch_size=64,
        n_epochs=4,
        gamma=0.99,
        ent_coef=0.01,
        # LSTM específico
        policy_kwargs=dict(
            lstm_hidden_size=64,
            n_lstm_layers=1,
        )
    )
    """
    return None

print("="*60)
print(f"  VARIANTE E: PPO CON MEMORIA (LSTM)")
print("="*60)

env = crear_entorno(env_name, wrapper="flat") # LSTM funciona mejor con MLP

model = RecurrentPPO(
    "MlpLstmPolicy",
    env,
    verbose=1,
    n_steps=128,
    batch_size=64,
    n_epochs=4,
    ent_coef=0.01,
)

callback = MinigridCallback()
model.learn(total_timesteps=timesteps, callback=callback, progress_bar=True)

env.close()
return model, callback

# Intentar entrenar con memoria
result_lstm = entrenar_con_memoria("DoorKey", timesteps=30000)

```

sb3-contrib no instalado. Instalar con: `pip install sb3-contrib`
 Requiere sb3-contrib. Ejemplo conceptual:

```

from sb3_contrib import RecurrentPPO

model = RecurrentPPO(

```

```

    "MlpLstmPolicy", # Política con LSTM
    env,
    verbose=1,
    n_steps=128,
    batch_size=64,
    n_epochs=4,
    gamma=0.99,
    ent_coef=0.01,
    # LSTM específico
    policy_kwargs=dict(
        lstm_hidden_size=64,
        n_lstm_layers=1,
    )
)

```

1.8 7. Evaluación Visual

```

[17]: def evaluar_visual(model, env_name="Empty", n_episodios=3):
    """Evalúa y visualiza el agente."""
    env_id = ENTORNOS.get(env_name, (env_name, "", ""))[0] if env_name in ENTORNOS else env_name
    env = gym.make(env_id, render_mode="rgb_array")
    env_wrapped = SimpleObsWrapper(env)

    for ep in range(n_episodios):
        obs, _ = env_wrapped.reset()
        frames = [env.render()]
        total_reward = 0
        done = False
        steps = 0

        while not done and steps < 100:
            action, _ = model.predict(obs, deterministic=True)
            obs, reward, term, trunc, _ = env_wrapped.step(action)
            frames.append(env.render())
            total_reward += reward
            done = term or trunc
            steps += 1

        # Mostrar algunos frames
        n_show = min(6, len(frames))
        indices = np.linspace(0, len(frames)-1, n_show, dtype=int)

        fig, axes = plt.subplots(1, n_show, figsize=(3*n_show, 3))
        for idx, ax in zip(indices, axes):
            ax.imshow(frames[idx])

```

```

ax.set_title(f"Step {idx}")
ax.axis('off')

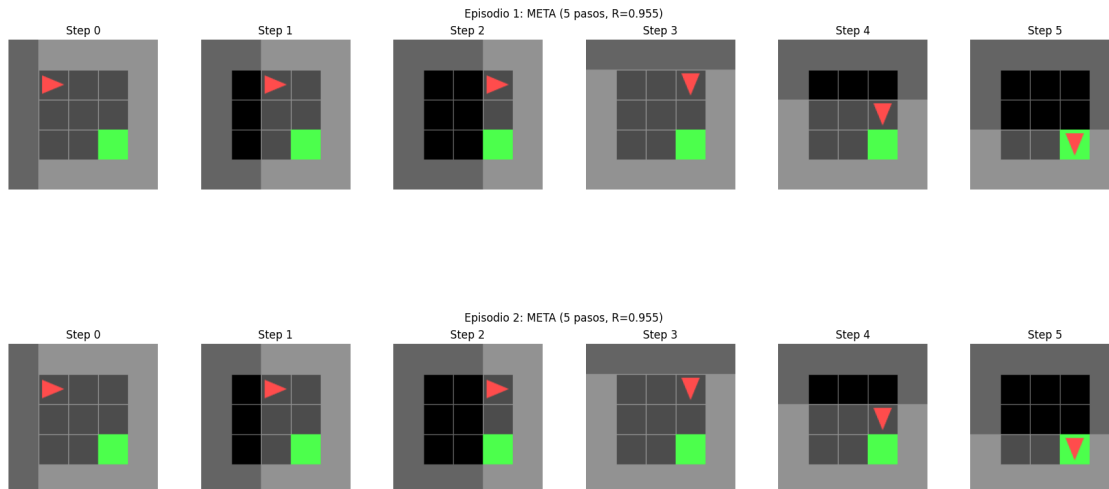
resultado = "META" if total_reward > 0 else "Fallo"
plt.suptitle(f"Episodio {ep+1}: {resultado} ({steps} pasos, R={total_reward:.3f})")
plt.tight_layout()
plt.show()

env.close()

# Evaluar el modelo entrenado
print("Evaluación del modelo entrenado:")
evaluar_visual(model, "Empty", n_episodios=2)

```

Evaluación del modelo entrenado:



1.9 8. Resumen y Conclusiones

1.9.1 Preguntas Respondidas

¿Por qué el agente solo ve 7×7 ? - MiniGrid implementa **observación parcial** para simular visión limitada - Esto crea desafíos de exploración y memoria que no existen con observación completa - Es más realista para aplicaciones robóticas

¿MLP o CNN? - **CNN** preserva estructura espacial y detecta patrones locales - **MLP** es más rápido pero pierde relaciones espaciales - Para observaciones $7 \times 7 \times 3$, CNN tiene ventaja significativa

¿Cómo resolver tareas que requieren memoria? - Usar arquitecturas recurrentes (LSTM/GRU) - RecurrentPPO de sb3-contrib - Alternativamente, agregar historial de observaciones (frame stacking)

1.9.2 Lecciones Aprendidas

Concepto	Hallazgo
Entropy coefficient	0.01-0.05 óptimo para recompensa sparse
Curriculum Learning	Ayuda significativamente en tareas difíciles
CNN vs MLP	CNN mejor para observaciones estructuradas
Generalización	Entrenar en múltiples entornos mejora transfer

1.9.3 Sigüientes Pasos

1. Probar entornos más difíciles (LavaCrossing, DistShift)
2. Implementar observation stacking para pseudo-memoria
3. Explorar attention mechanisms para observación parcial

1.10 9. Referencias

- [MiniGrid Documentation](#)
- [Stable-Baselines3](#)
- [PPO Paper](#)
- [Curriculum Learning Survey](#)

1.11 Variantes de Entrenamiento — MiniGrid

Las variantes en MiniGrid exploran cómo **representar la observación**: la misma imagen $7 \times 7 \times 3$ puede procesarse de formas muy distintas.

Variante	Red	Observación	Estructura espacial
A	MLP	Vector aplanado (147D)	No aprovecha
B	CNN (<i>actual</i>)	Imagen $7 \times 7 \times 3$	Sí aprovecha
C	CNN + Curriculum	Múltiples entornos	Sí + progresión

Observación MiniGrid: el agente ve una ventana parcial 7×7 centrada en él. Cada celda tiene 3 valores: tipo de objeto, color, estado.

1.11.1 Variante A — MLP + Observación Aplanada

```
python minigrid_navegacion.py --variant flat
```

La imagen $7 \times 7 \times 3$ se aplanar a un vector de **147 valores** y se pasa directamente a una red densa (MLP).

Desventaja fundamental: el MLP no sabe que el valor en la posición [3,4] está *al lado* del valor en [3,5]. Pierde la información espacial.

```
Imagen 7x7x3:          Vector 147D:
                        [0.0, 1.0, 2.0, 0.0, 1.0, ...]
... . . . . . flatten
```

```
. A . . . .      147 valores sin estructura
. . . G . .      espacial
```

Cuándo funciona: entornos pequeños y simples donde la posición relativa importa poco.

```
[18]: # Variante A: MLP + Observación Aplanada
# from minigrid_navegacion import entrenar_flat
# model, callback = entrenar_flat(env_name="Empty", timesteps=30000,
# ↪algorithm="PPO")

print("Variante A: MLP + Observación Aplanada")
print()
flat_code = """
# FlatObsWrapper: aplanar la imagen a vector
class FlatObsWrapper(gym.ObservationWrapper):
    def __init__(self, env):
        obs_shape = env.observation_space['image'].shape # (7, 7, 3)
        self.observation_space = gym.spaces.Box(
            low=0, high=255,
            shape=(np.prod(obs_shape),), # 147
            dtype=np.float32
        )

    def observation(self, obs):
        return obs['image'].flatten().astype(np.float32) / 255.0

# Red: MLP simple
PPO(
    "MlpPolicy", # Red densa
    env,
    policy_kwargs={"net_arch": [256, 256]},
)
"""
print(flat_code)
print("Tamaño de entrada: 7×7×3 = 147 valores")
print("Red: [147 → 256 → 256 → 6 acciones]")
```

Variante A: MLP + Observación Aplanada

```
# FlatObsWrapper: aplanar la imagen a vector
class FlatObsWrapper(gym.ObservationWrapper):
    def __init__(self, env):
        obs_shape = env.observation_space['image'].shape # (7, 7, 3)
        self.observation_space = gym.spaces.Box(
            low=0, high=255,
            shape=(np.prod(obs_shape),), # 147
```



```

        dtype=np.float32
    )

    def observation(self, obs):
        return obs['image'].flatten().astype(np.float32) / 255.0

# Red: MLP simple
PPO(
    "MlpPolicy",                # Red densa
    env,
    policy_kwargs={"net_arch": [256, 256]},
)

```

Tamaño de entrada: $7 \times 7 \times 3 = 147$ valores
 Red: $[147 \rightarrow 256 \rightarrow 256 \rightarrow 6 \text{ acciones}]$

1.11.2 Variante B — CNN Personalizada (*implementación actual*)

`python minigrid_navegacion.py --variant cnn`

La imagen $7 \times 7 \times 3$ se procesa con **convoluciones** que respetan la estructura espacial. Cada filtro convolucional “mira” una zona de la imagen.

Arquitectura MinigridCNN:

Entrada: $7 \times 7 \times 3$

- Conv2d(3→16, kernel=2) → $6 \times 6 \times 16$
- Conv2d(16→32, kernel=2) → $5 \times 5 \times 32$
- Conv2d(32→64, kernel=2) → $4 \times 4 \times 64$
- Flatten → 1024
- Linear(64) → 64 features
- política PPO/DQN

Ventaja vs MLP: los filtros convolucionales aprenden a detectar patrones visuales locales (paredes, puertas, la posición relativa del agente).

```

[19]: # Variante B: CNN personalizada (default)
# from minigrid_navegacion import entrenar_minigrid
# model, callback = entrenar_minigrid(env_name="DoorKey", timesteps=50000,
# ↪ algorithm="PPO")

print("Variante B: CNN personalizada (MinigridCNN)")
print()
cnn_code = """
class MinigridCNN(BaseFeaturesExtractor):
    def __init__(self, observation_space, features_dim=64):
        self.cnn = nn.Sequential(
            nn.Conv2d(3, 16, kernel_size=2),    #  $7 \times 7 \times 3 \rightarrow 6 \times 6 \times 16$ 
            nn.ReLU(),
            nn.Conv2d(16, 32, kernel_size=2),   #  $6 \times 6 \times 16 \rightarrow 5 \times 5 \times 32$ 

```

```

        nn.ReLU(),
        nn.Conv2d(32, 64, kernel_size=2), # 5×5×32 → 4×4×64
        nn.ReLU(),
        nn.Flatten(), # → 1024
    )
    self.linear = nn.Sequential(
        nn.Linear(1024, features_dim), # → 64
        nn.ReLU()
    )

    def forward(self, x):
        x = x.permute(0, 3, 1, 2) # (B,H,W,C) → (B,C,H,W)
        return self.linear(self.cnn(x))

# Usar en PPO:
policy_kwargs = {
    "features_extractor_class": MinigridCNN,
    "features_extractor_kwargs": {"features_dim": 64}
}
PPO("CnnPolicy", env, policy_kwargs=policy_kwargs, ...)
"""
print(cnn_code)

```

Variante B: CNN personalizada (MinigridCNN)

```

class MinigridCNN(BaseFeaturesExtractor):
    def __init__(self, observation_space, features_dim=64):
        self.cnn = nn.Sequential(
            nn.Conv2d(3, 16, kernel_size=2), # 7×7×3 → 6×6×16
            nn.ReLU(),
            nn.Conv2d(16, 32, kernel_size=2), # 6×6×16 → 5×5×32
            nn.ReLU(),
            nn.Conv2d(32, 64, kernel_size=2), # 5×5×32 → 4×4×64
            nn.ReLU(),
            nn.Flatten(), # → 1024
        )
        self.linear = nn.Sequential(
            nn.Linear(1024, features_dim), # → 64
            nn.ReLU()
        )

    def forward(self, x):
        x = x.permute(0, 3, 1, 2) # (B,H,W,C) → (B,C,H,W)
        return self.linear(self.cnn(x))

# Usar en PPO:
policy_kwargs = {

```

```

    "features_extractor_class": MinigridCNN,
    "features_extractor_kwargs": {"features_dim": 64}
}
PPO("CnnPolicy", env, policy_kwargs=policy_kwargs, ...)

```

1.11.3 Variante C — Curriculum Learning

`python minigrid_navegacion.py --curriculum`

El agente aprende habilidades de navegación progresivamente:

Nivel	Entorno	Habilidad nueva
1	Empty-5x5	Ir a la meta (sin obstáculos)
2	Empty-8x8	Planificar rutas más largas
3	FourRooms	Navegar entre habitaciones
4	DoorKey	Recoger llave, abrir puerta, ir a meta
5	LavaCrossing	Evitar obstáculos de lava

Concepto clave: DoorKey es muy difícil de aprender desde cero porque el agente debe: 1. Encontrar la llave 2. Recogerla 3. Encontrar la puerta 4. Abrirla 5. Llegar a la meta

Con curriculum, el agente ya sabe navegar (nivel 1-3) y solo necesita aprender la secuencia llave-puerta.

```

[20]: # Variante C: Curriculum Learning
# from minigrid_navegacion import curriculum_minigrid
# model, historial = curriculum_minigrid(timesteps_por_nivel=20000,
↳ algorithm="PPO")
# Genera: minigrid_curriculum.png

print("Variante C: Curriculum Learning (5 niveles)")
print()
curriculum_code = """
curriculum = [
    ("Empty",          "Navegar a meta - trivial"),
    ("Empty8",         "Meta más lejos"),
    ("FourRooms",      "Navegar entre habitaciones"),
    ("DoorKey",        "Llave → puerta → meta"),
    ("LavaCrossing",   "Evitar lava"),
]

# Transferencia entre niveles:
if nivel == 0:
    model = PPO("CnnPolicy", env, policy_kwargs=cnn_kwargs, ...)
else:
    model.set_env(env) # Mismo modelo, entorno más complejo
    # Los pesos CNN se mantienen (ya sabe 'ver')

```

```

# Solo necesita aprender nuevos comportamientos

model.learn(timesteps_por_nivel, reset_num_timesteps=(nivel==0))
"""
print(curriculum_code)
print("Métricas de seguimiento:")
print("  - Recompensa por episodio (progresión)")
print("  - Tasa de éxito acumulada (% episodios que llega a la meta)")

```

Variante C: Curriculum Learning (5 niveles)

```

curriculum = [
    ("Empty", "Navegar a meta - trivial"),
    ("Empty8", "Meta más lejos"),
    ("FourRooms", "Navegar entre habitaciones"),
    ("DoorKey", "Llave → puerta → meta"),
    ("LavaCrossing", "Evitar lava"),
]

# Transferencia entre niveles:
if nivel == 0:
    model = PPO("CnnPolicy", env, policy_kwargs=cnn_kwargs, ...)
else:
    model.set_env(env) # Mismo modelo, entorno más complejo
    # Los pesos CNN se mantienen (ya sabe 'ver')
    # Solo necesita aprender nuevos comportamientos

model.learn(timesteps_por_nivel, reset_num_timesteps=(nivel==0))

Métricas de seguimiento:
- Recompensa por episodio (progresión)
- Tasa de éxito acumulada (% episodios que llega a la meta)

```

1.11.4 Comparativa: MLP vs CNN vs Curriculum

Aspecto	A: MLP+Flat	B: CNN	C: CNN+Curriculum
Parámetros	Menos	Más	Más
Entiende espacial	No	Sí	Sí
Tasa de éxito (Easy)	Media	Alta	Alta
Tasa de éxito (DoorKey)	Baja	Media	Alta
Timesteps necesarios	Menos	Más	Muchos

Lección principal: en entornos con estructura espacial (laberintos, cuadrículas), las CNN son casi siempre superiores a los MLP. El curriculum multiplica ese efecto en entornos difíciles.

Experimento recomendado: entrenar Variante A y B en DoorKey con los mismos timesteps y comparar la tasa de éxito final.